

Lecture 5

Noncontiguous Memory Allocation

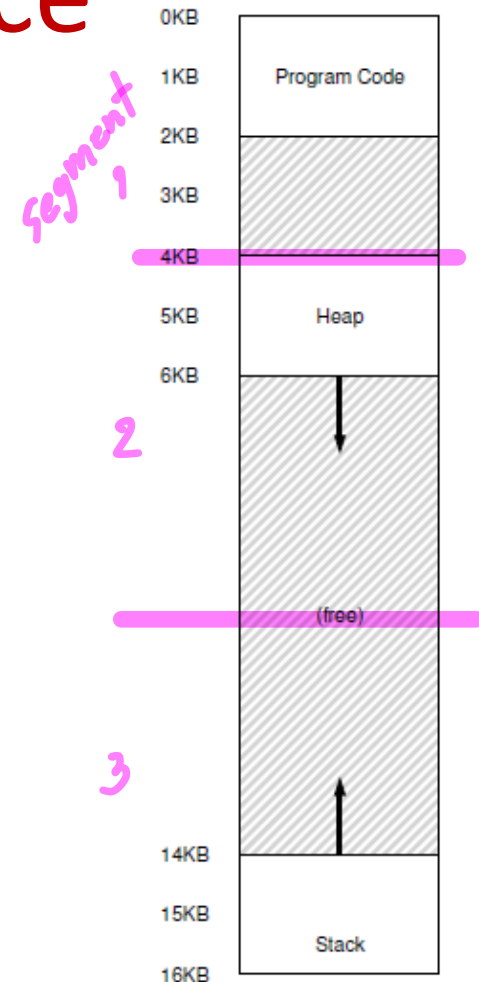
Previously..

- We know disk and memory are different
- Era of programmable machine and time sharing machine
- Physical and logical address (translation)
- Memory allocation: Contiguous allocation
 - Single partition (use base and bound register)
 - Multiple partition
 - Fixed-size of partition
 - Static relocation (load-time address binding)
 - Dynamic relocation (run-time address binding: when context switch occur)
 - Flexible-size of partition

Process Address Space

- Problems of Contiguous Memory Allocation

- The **entire address space** of each process is stored in physical memory
- However, the address space has a lot of **unused space** between the heap and the stack which takes up physical memory and is wasted
- There are also problems in using both fixed-size partitions and flexible-size partitions with Contiguous Memory Allocation, as discussed last time
- Noncontiguous Memory Allocation attempts to solve some of these problems



code heap stack
מזכרון פיזי
מזכרון לוגי

מזכרון פיזי (paging)

Segmentation Scheme vs Paging Scheme

Logical Address Space

Segmentation Scheme

mem כחלק מ: זכרון מוגבל

Code
Heap
Stack
etc.

- The logical address space is divided into a number of parts, called **segments**, of **unequal size** (but there is a limit to the size)
 - A segment is just a contiguous portion of the address space of a particular length
- We extend the Boundary Scheme to have a **Base and Bounds register** pair **per segment**
- Initially, we will assume there are three logical segments: **code**, **stack** and **heap** segments
 - In this case, the Memory Management Unit (MMU) now includes three pairs of Base and Bounds registers

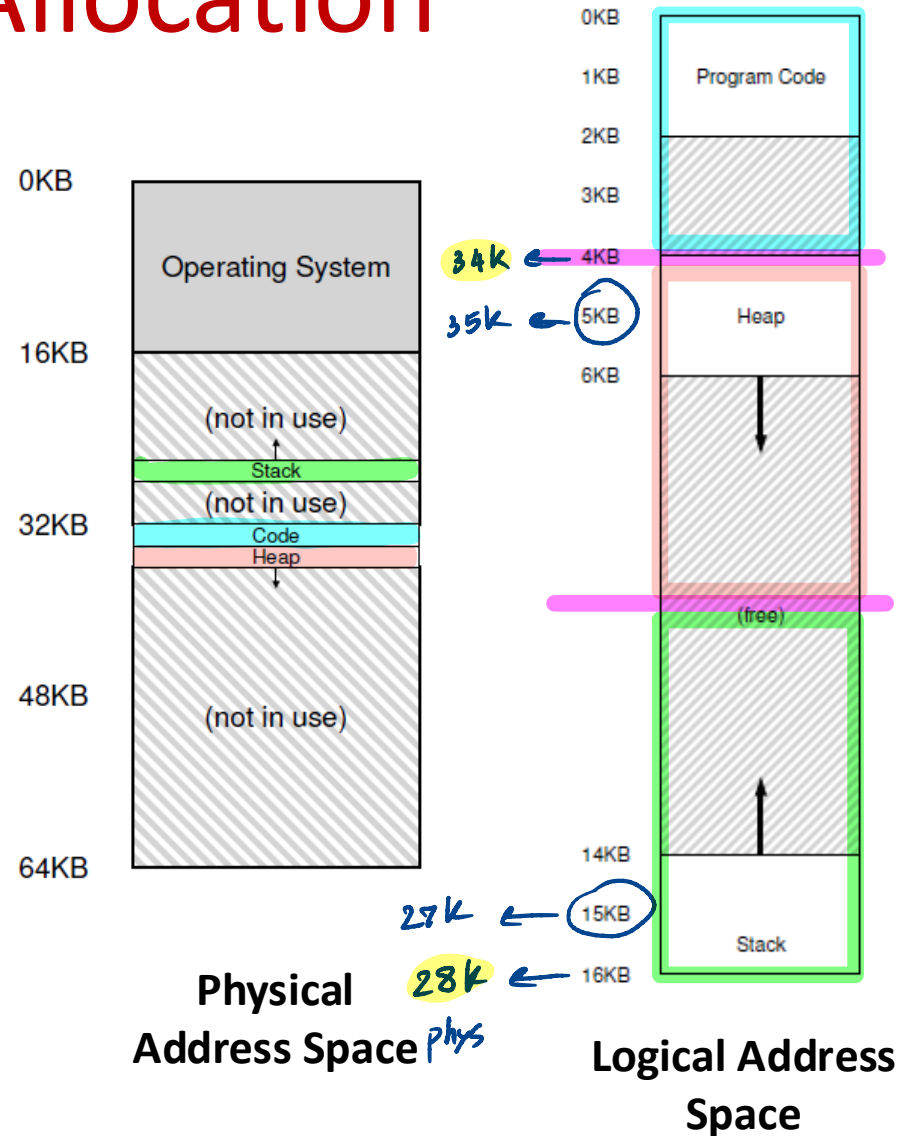
logical @ address

Segment Allocation

- Segmentation allows each of the segments to be placed in a different part of physical memory
- Assume Base and Bound register pairs for the three segments are:

Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K

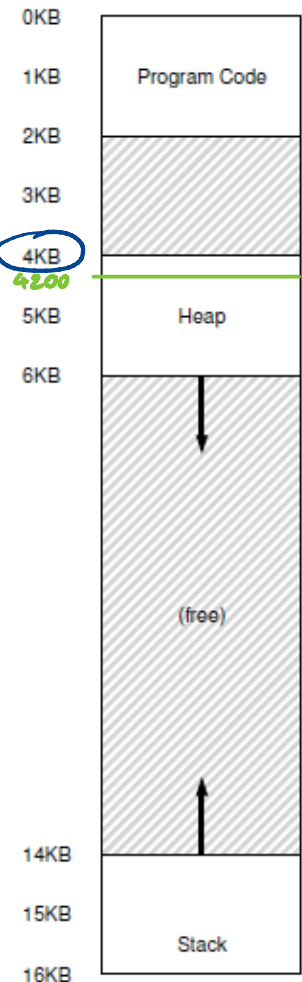
- Avoids storing unused parts of the address space in physical memory



Address Translation

Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K

- Suppose we have a logical address of 4200, which is an address in the heap
- Cannot simply add the heap base value to the logical address ($4200 + 34\text{KB}$ is the wrong physical address)
- In the **logical** address space of the previous example, suppose the heap starts at 4KB
- Instead we want to add the heap base value to the offset in the heap ($4200 - 4096 = 104$ and $104 + 34\text{KB}$ is the correct physical address)
- But how do we know **which segment** a logical address belongs to?

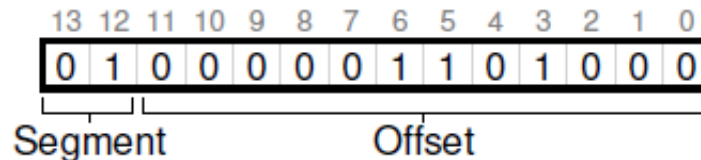


Logical Address Space

Segment and Offset

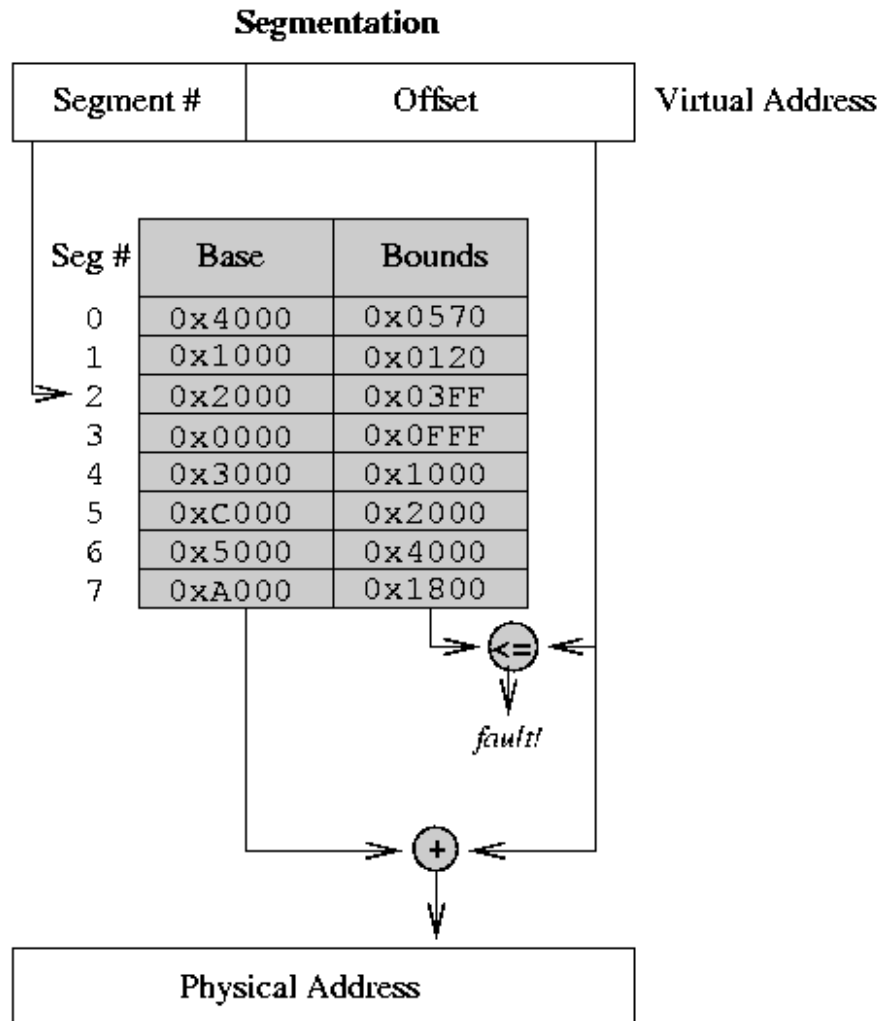
- Logical Address

- Organize the logical address space so that the top few bits of the logical address denote the segment
 - With three segments, two bits are enough, e.g. code segment = 00, heap segment = 01, etc.
- The remaining bits denote the offset in the segment
- The logical address is then as follows



- However, the stack grows backward in the logical address space
 - Have an extra bit to denote the direction of growth

Segmentation with Segment Table



- Supporting a number of segments makes use of a **Segment Table**
 - Each entry is a **Base and Bounds pair**
- A logical address consists of two parts: Segment and Offset
- High bits of address select Segment, low bits select the Offset
- Segment Table per Process**
- Where is the Segmentation table stored? Ans RAM

Access 2 נתון → נתון 1 : מזהה את ה-segment table
 → נתון 2 : מזהה את ה-data

Segment 140: — 1023 translate 100000, 10230000. (table 1000000)
အကယ်၍ search တစ်ခု row 1023 ကို table 1000000 ကို offset 1000000

Issues in Address Translation

- Extra Registers for Specific Tasks

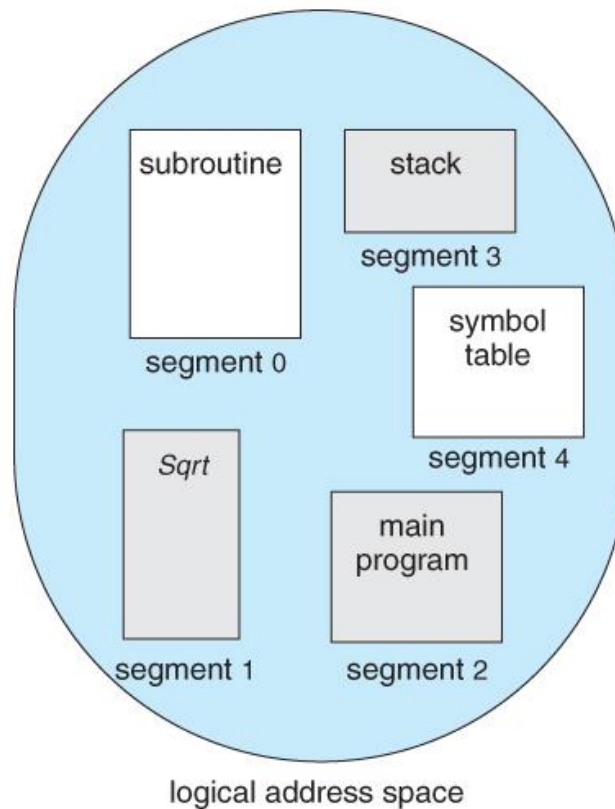
- Define the direction of memory growth (set when segment grows in the positive direction)
- Control the permission of segments (read-write-execute) to allow possible sharing of segments between address spaces

Segment	Base	Size	Grows Positive?	Protection
Code	32K	2K	1	Read-Execute
Heap	34K	2K	1	Read-Write
Stack	28K	2K	0	Read-Write

- Fine- vs. Coarse-Grained Segmentation

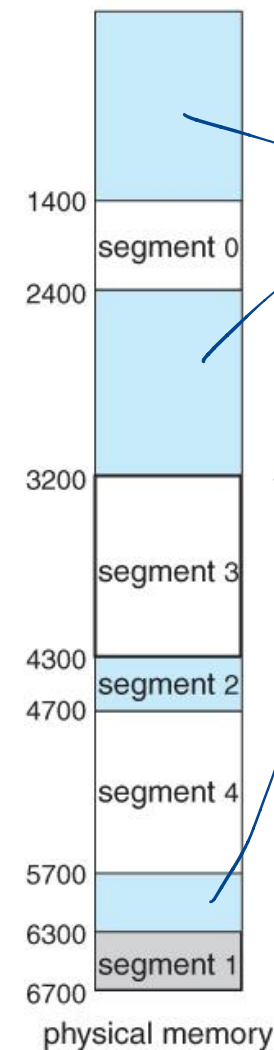
- We may have a few coarse-grain segments (e.g. three) or a large number of fine-grain segments

Example of Fine-Grained Segmentation



	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table



non
External
fragmentation

Memory Management

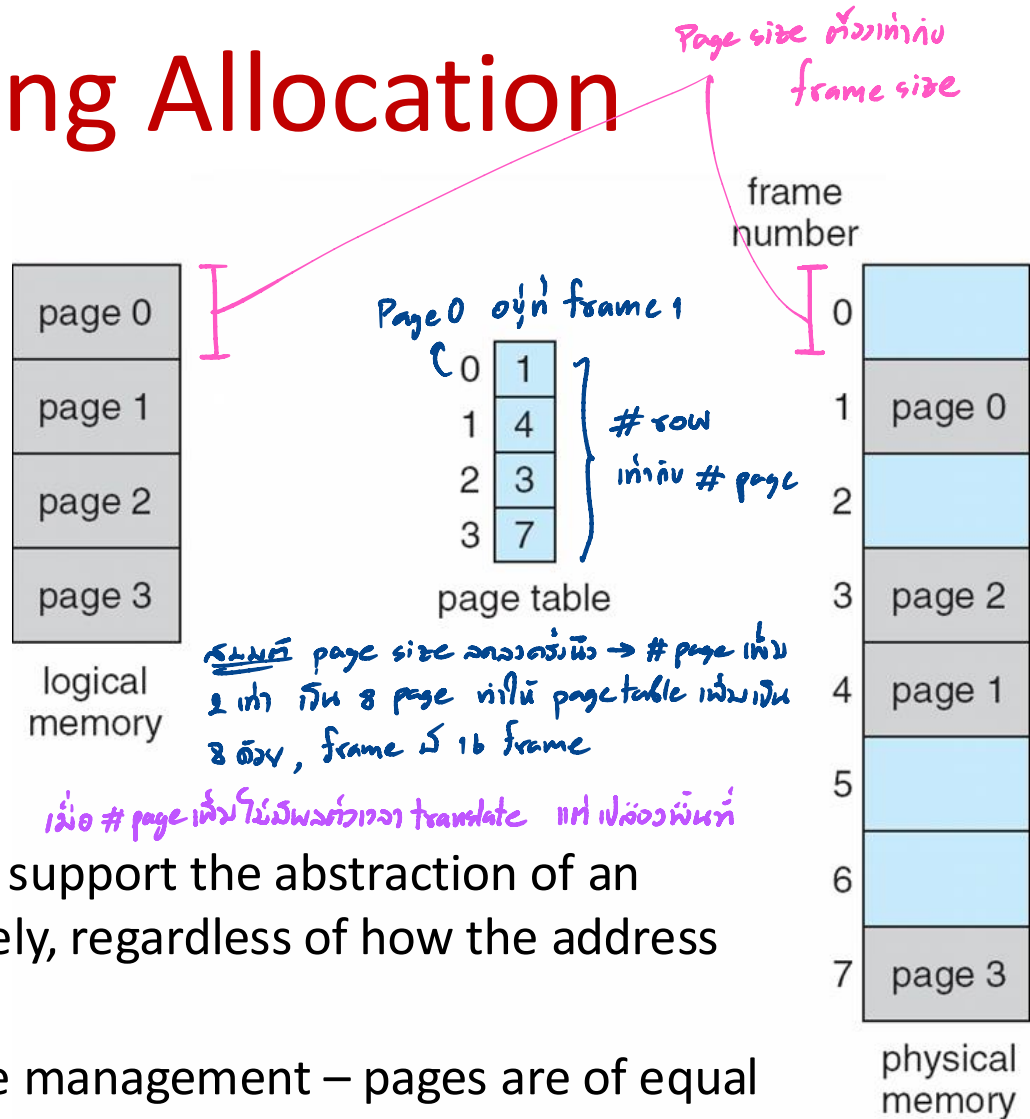
- Given a new address space, we need to find space in physical memory for the segments (which are of unequal size)
 - OS keeps a free list of the unused space (holes), and use an allocation strategy
 - *First-fit strategy*: The segment is placed in the first hole that is of sufficient size (has the advantage of speed)
 - *Best-fit strategy*: The segment is placed in the hole that leaves the least unused space (tries to reduce wasted space)
 - *Worst-fit strategy*: The segment is placed in the hole that leaves the most unused space (tries to avoid leaving small holes that can arise with the best-fit strategy)
 - As external fragmentation is possible, it may also be necessary to use coalescing or compaction

Paging Scheme

- The **logical address** space is divided into a number of parts, called ***pages***, which are of **equal size**
- The **physical memory** is divided into a number of parts, called ***frames***, which are of equal size and the same size as the page size
 - Each frame can contain a single page
- Typical page sizes range from 512 to 16K Bytes
- Each process has a page table that is used to translate logical addresses to physical addresses
- Paging attempts to address the problem of external fragmentation that occurs with segmentation

Paging Allocation

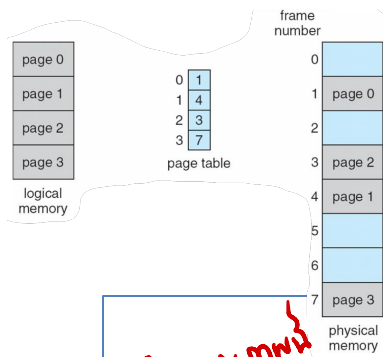
The page table is a per-process data structure



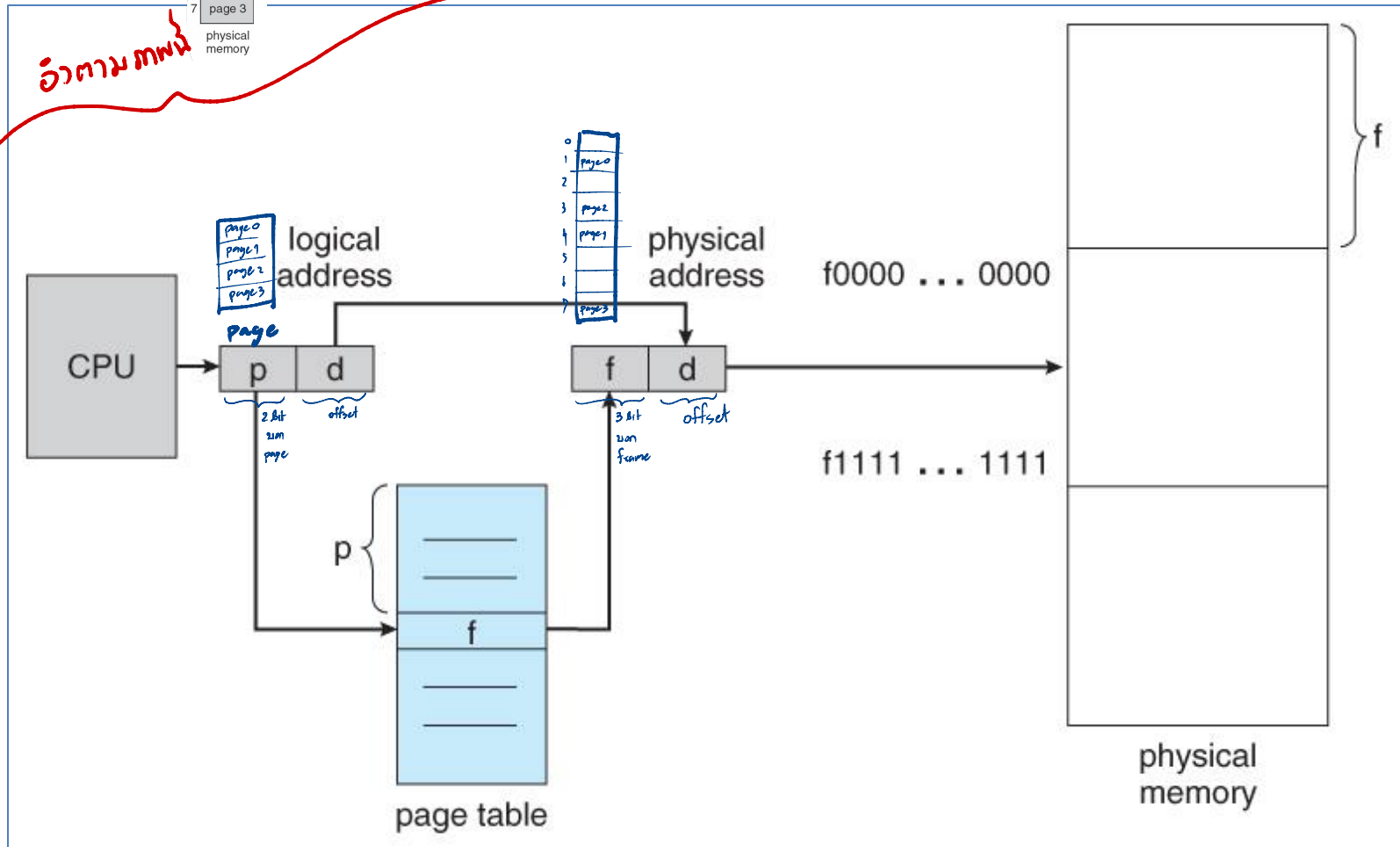
- Advantages of Paging**

- Flexibility – Paging can support the abstraction of an address space effectively, regardless of how the address space is used
- Simplicity of free space management – pages are of equal size, so can use a simple free list of all free frames

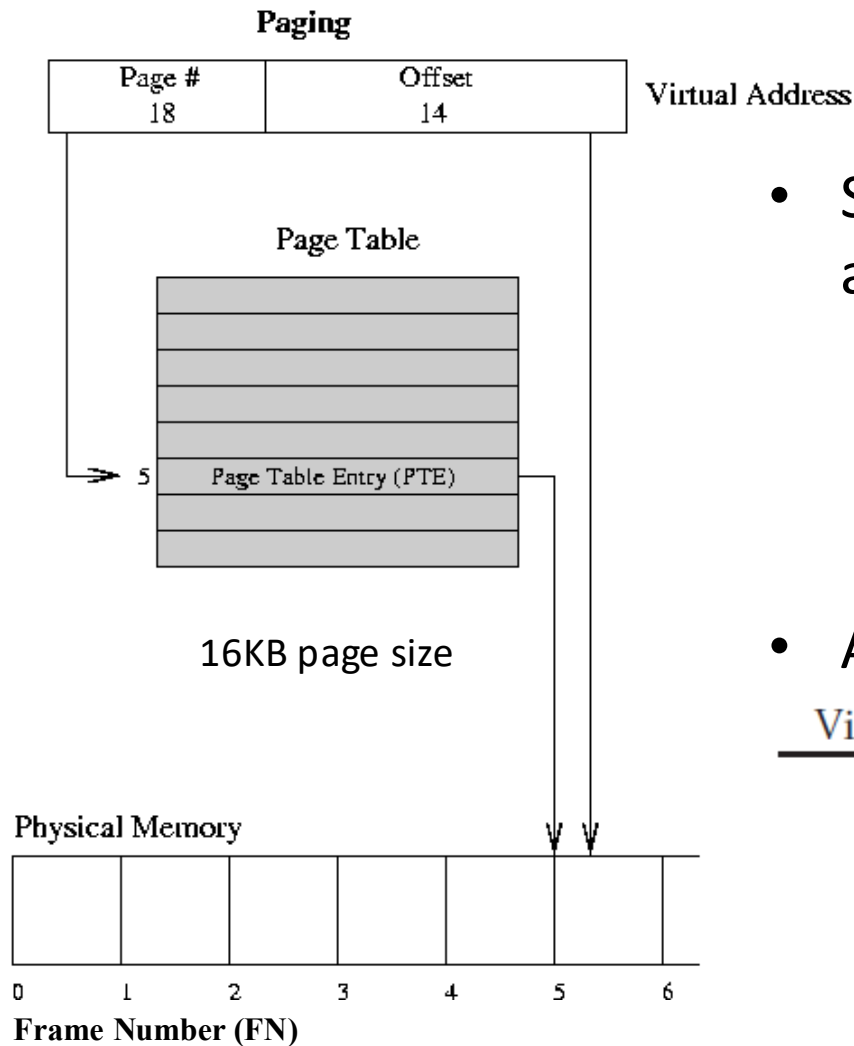
page size ↓ → #page ↑ → page table ใหญ่ขึ้น



Basic Page Table



Address Translation



- Similar to segmentation, a logical address is divided into two parts:
 - High bits are used to define the logical page number (PN)
 - Low bits are the offset value within the page
- Another Example of Page Table

Virtual Page Number	Physical Page Frame
0	3
1	7
2	5
3	2

How Large is the Page Table?

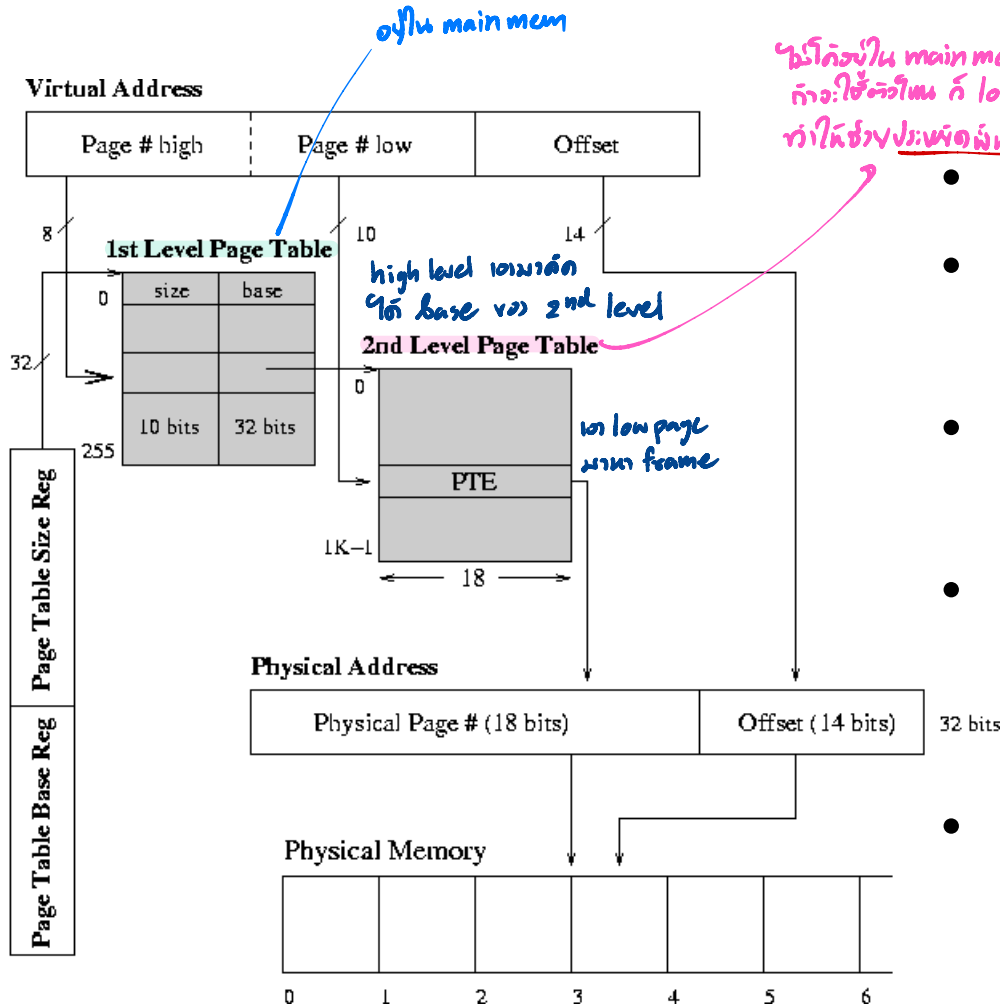
- Consider a 32 bit logical address where the 20 MSBs define page number and the 12 LSBs define page offset (4KB page)
- Assume 32 bits to keep all the ^{frame number}FNs in physical memory
- Use 1 bit for a valid bit (space used)
- So, 2^{20} is the number of PTEs and the size of the page table is $2^{20} * 33$ (~34 MByte)
- This is just one process!
- As the tables are so large, they are held in kernel memory – a **Page Table Base Register** contains the physical address of the start of the page table and a **Page Table Size Register** its size

V	Physical Page
0	32 bits
1	
2	
	⋮
$2^{20}-1$	

100 processes → 36B

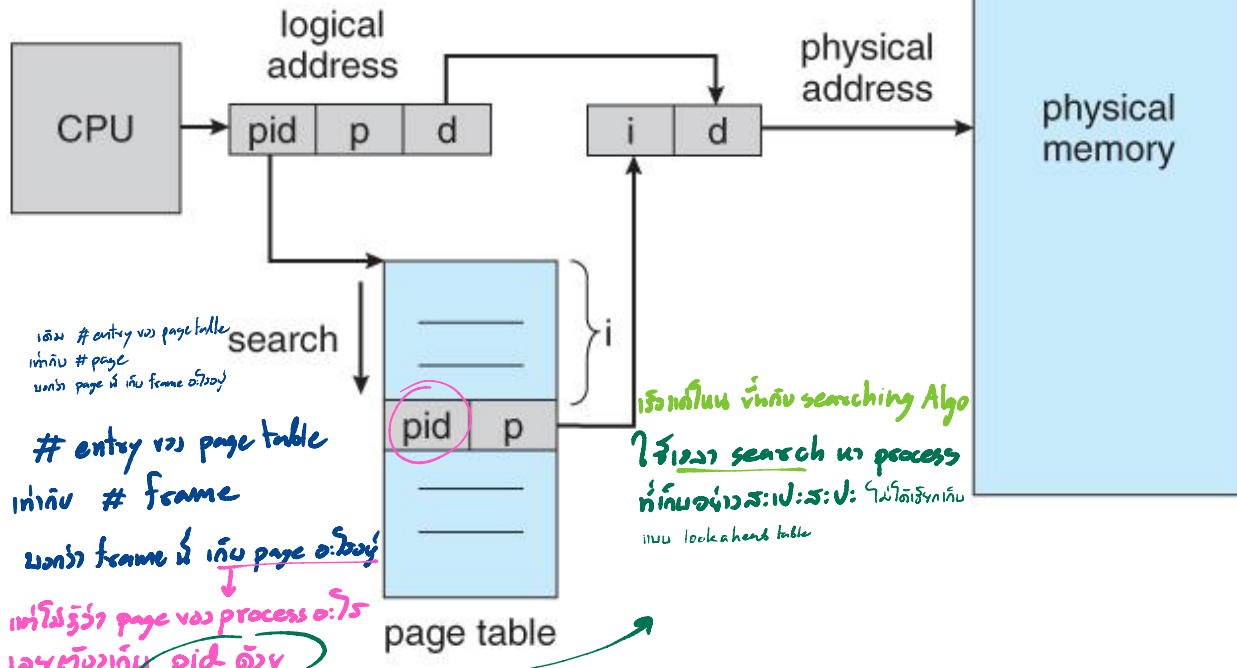
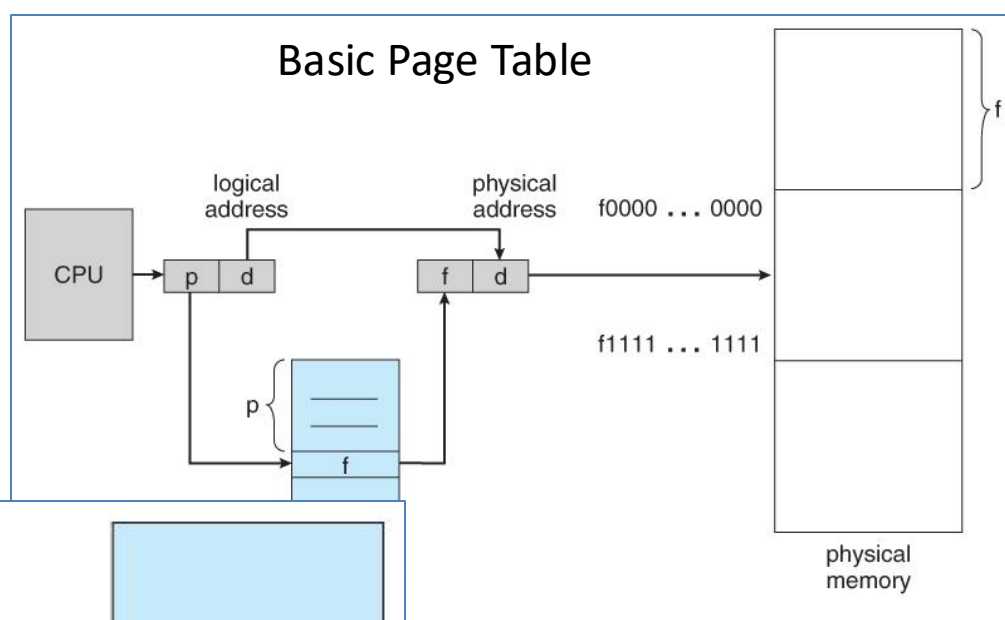
1000 processes → 34 GB

Two Level Paging Scheme



- The page table is itself paged
- Page # high is an index into the 1st level page table
- Page # low is an index into the 2nd level page table
- Reduces the size as it is not necessary for all 2nd level page tables to be present
- Desirable to fit a 2nd level page table into one page memory

Inverted Page Table



- Search table for entry corresponding to process and logical page
- Use index of entry as page frame number in physical memory

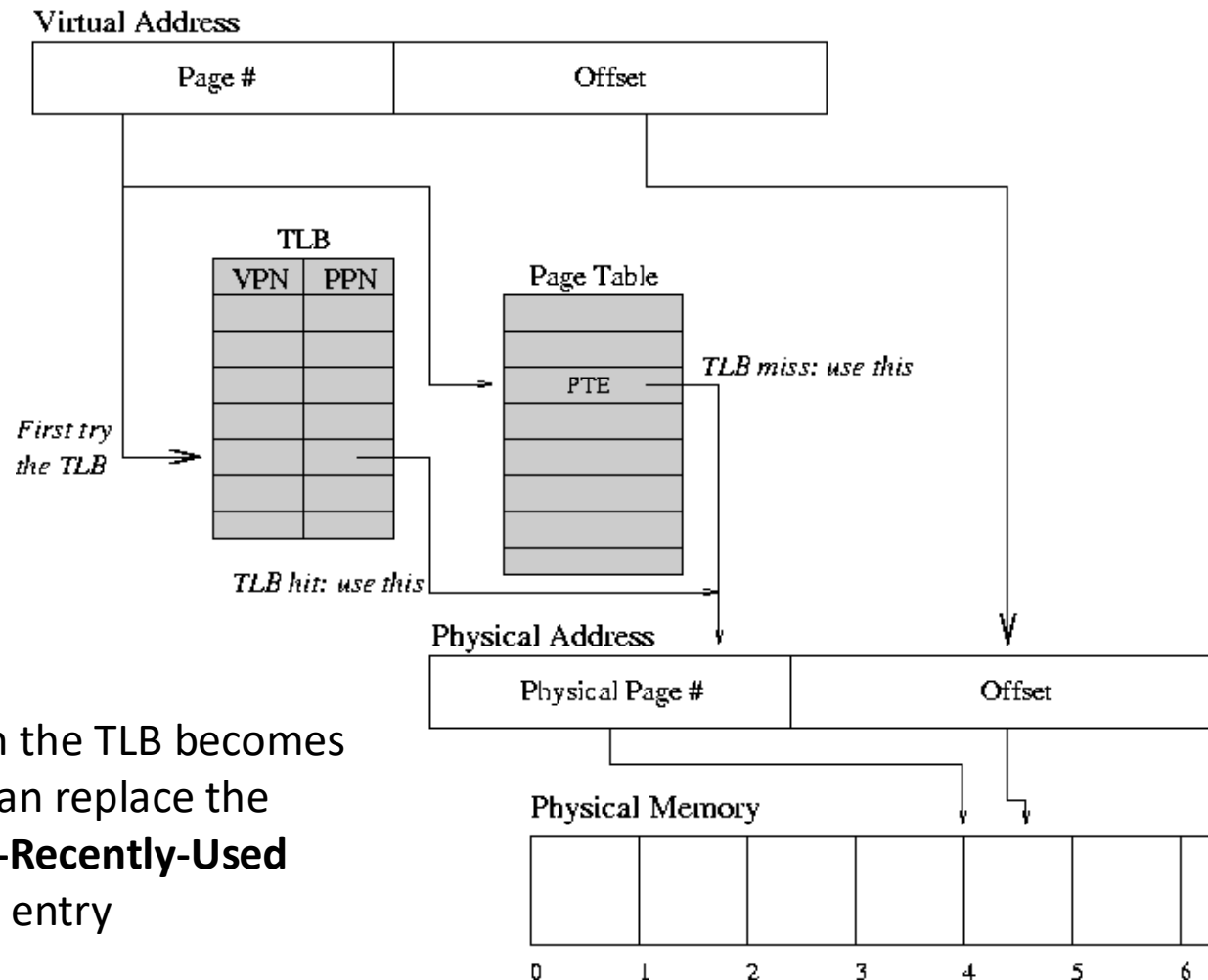
Inverted Page Table

- Reduces size of page table based on the idea that the number of frames is smaller than the number of pages
- The inverted page table has one entry for each page frame of physical memory
- An entry consists of the logical page number stored in that page frame and the process identifier (pid) of the process that owns that page
- However, it **increases the address translation time** as it is necessary to **search** the table when a page reference occurs

Paging with Fast Translation

- An overhead of the paging scheme is the efficiency of access
 - The page table is so large that it has to be kept in main memory
 - A linear page table therefore requires an extra memory lookup for each logical address generated by the program
 - Techniques to reduce the size of the page table involve extra memory lookups and increase the translation time even further
 - There is a time-space tradeoff
- How can we speed up address translation?
 - *Translation Lookaside Buffer* – special hardware that is part of the MMU

Translation Lookaside Buffer



When the TLB becomes full, can replace the **Least-Recently-Used (LRU)** entry

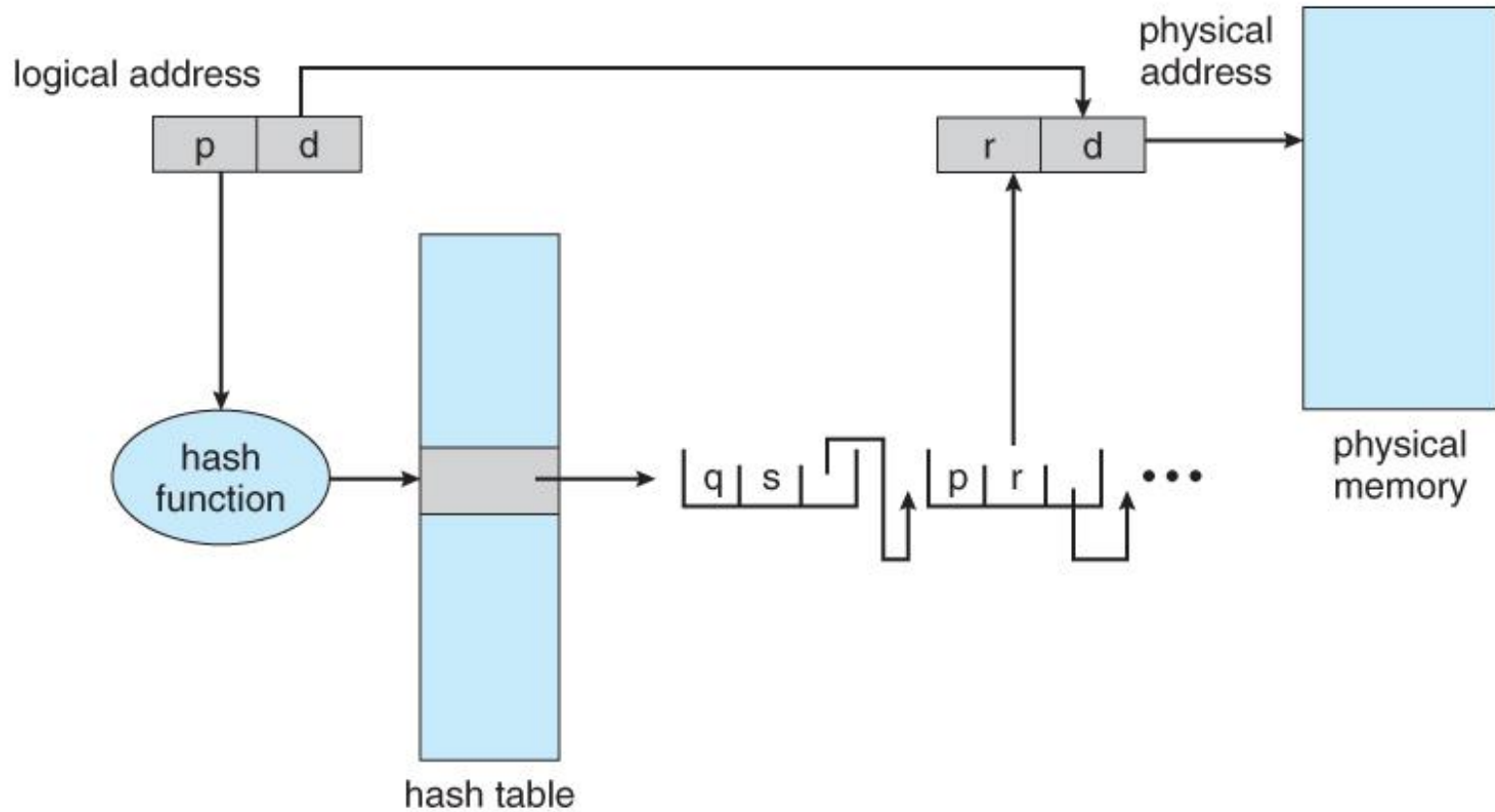
Translation Lookaside Buffer

- The TLB is simply a hardware cache of popular logical page number to physical page frame translations
 - Fully associative – hardware will search entire TLB in parallel
- Based on idea of locality of reference
- The hardware looks in TLB first
 - If it has the required page, the translation is very quick
 - Otherwise, the page table is accessed in the normal way and the translation stored in the TLB
- The TLB contains address translations for the current process
 - The TLB is flushed (entries marked invalid) on a context switch

Hashed Page Table

- The logical page number is hashed into a hashed page table
 - Each entry contains a chain of elements hashed to that entry
- Each element consists of the logical page number, the mapped page frame, and a pointer to the next element
- The chain is searched by logical page number looking for a match
 - If a match is found, the corresponding physical page frame is extracted
- Reduces the size of the page table as it is not necessary to have an entry for every logical page (only valid pages)
- Common in address spaces > 32 bits

Hashed Page Table



Page Size

- **Small Page Size** *Page size ↓ → internal frag ↓, but page table size ↑, external fragmentation*
 - Reduces internal fragmentation
 - Can reduce the amount of memory required to contain a process's working set
 - More memory is available to other processes
- **Large Page Size**
 - Reduces wasted memory from table fragmentation
 - Enables each TLB entry to map a larger region of memory, improving performance
 - Reduces the number of I/O operations the system performs to load a process's working set into memory

Segmentation vs. Paging

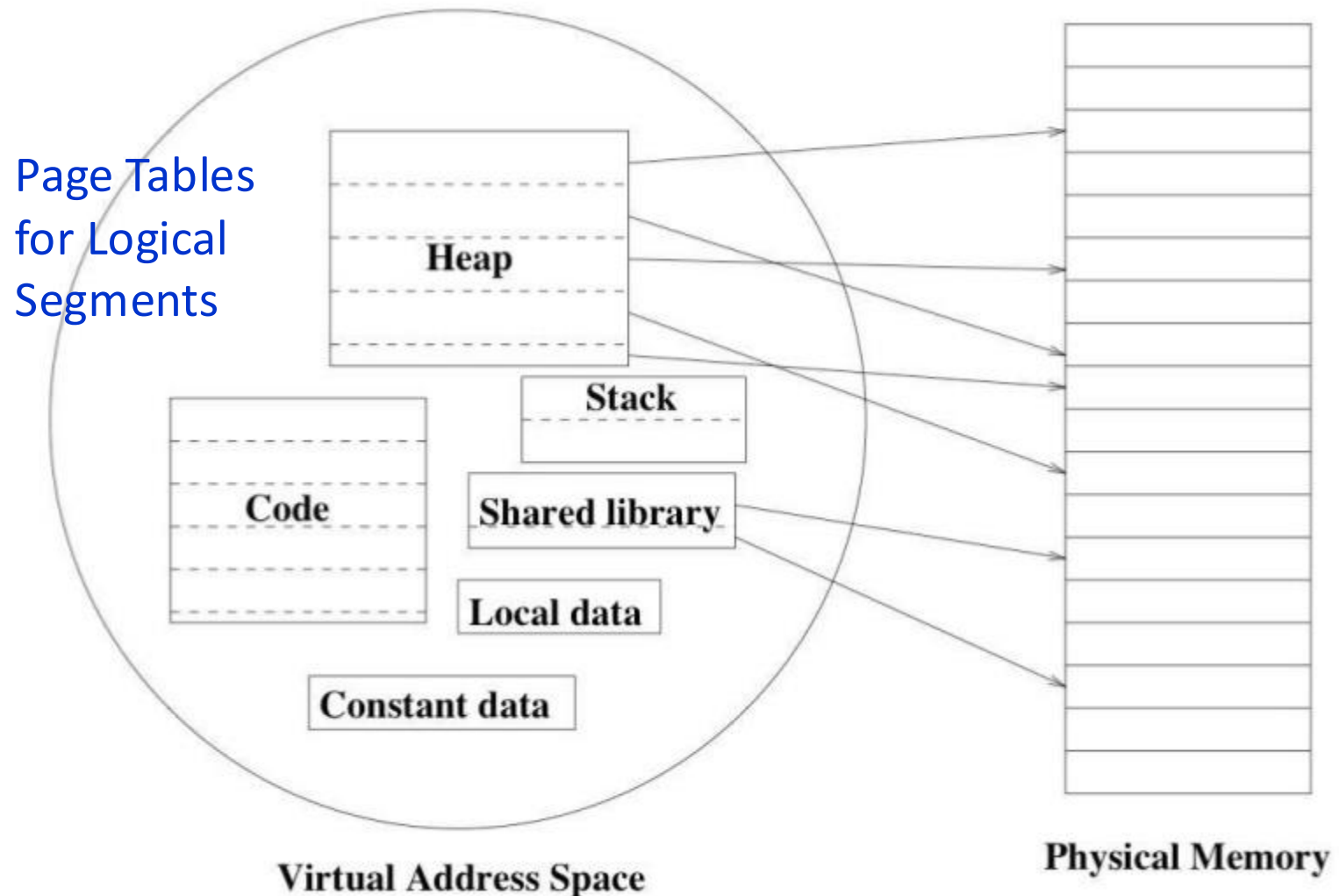
- Segmentation Advantages
 - No internal fragmentation
 - Segment table is very small when compared with page table
- Paging Advantages
 - No external fragmentation
 - Flexible scheme and simple memory management
- Segmentation Disadvantages
 - External fragmentation
 - Memory management not easy as segments of unequal size
- Paging Disadvantages
 - Internal fragmentation (especially if page size is large)
 - Page table consumes a lot of memory and has memory reference overhead

✓ 9504
only

Hybrid Scheme

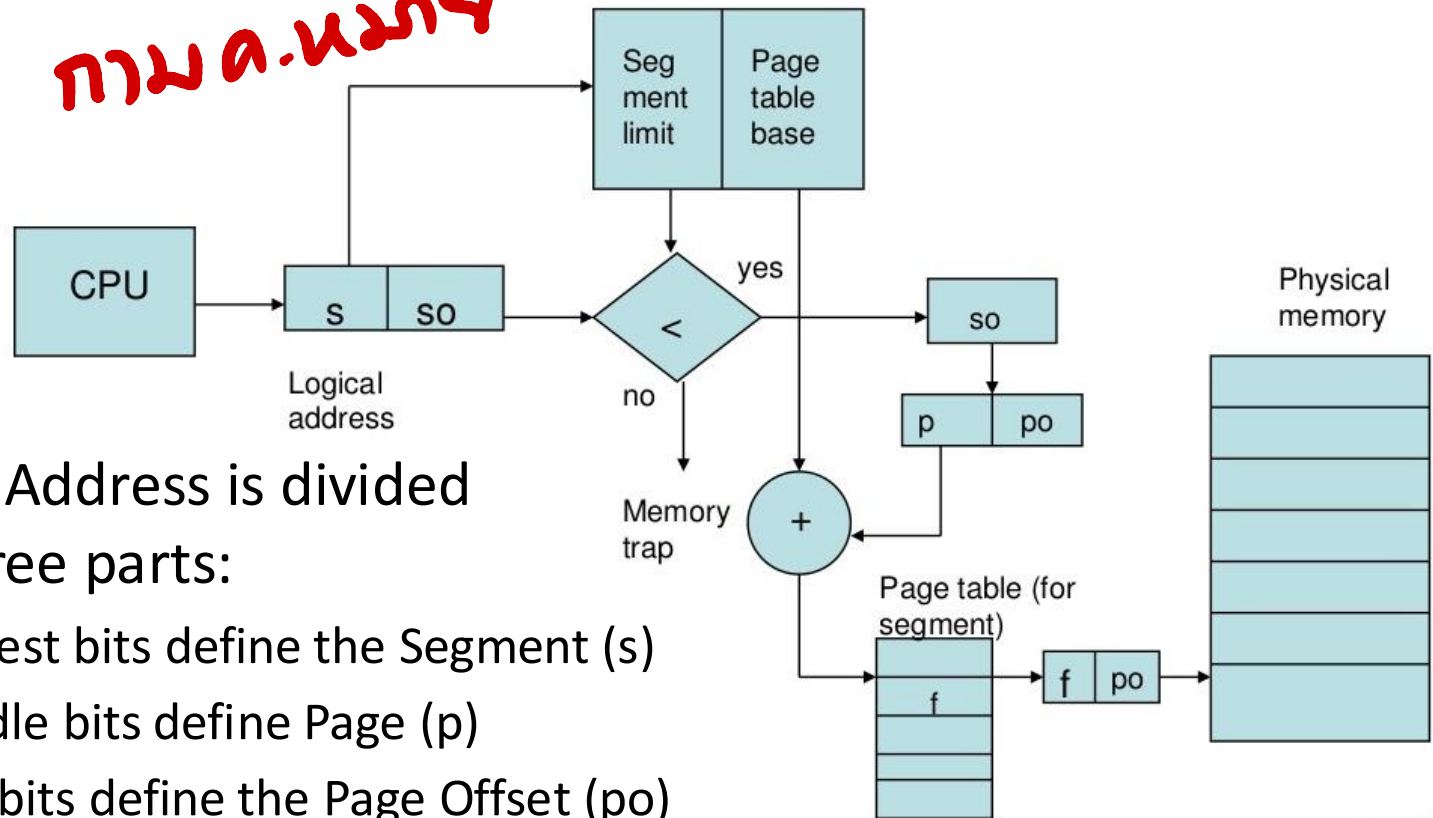
- Segmentation + Paging Scheme
 - Address space is divided into logical segments, where each logical segment is divided into a number of pages
 - Instead of having a single page table for the entire address space, have one page table per logical segment
 - Have a Base and Bounds register pair per segment as before
 - The Base register holds the physical address of the page table for the segment
 - The Bounds register is used to indicate the size of the page table
 - A Logical Address is divided into three parts:
 - Segment Number, Page Number and Page Offset

Segmentation + Paging Scheme



Address Translation with Segment Table and Page Table

כחול
סגור



- Logical Address is divided into three parts:
 - Highest bits define the Segment (s)
 - Middle bits define Page (p)
 - Low bits define the Page Offset (po)
- A process has one segment table, but can have several page tables, depending on the number of segments

Summary of Hybrid Scheme

- **Advantages**

- Significant **memory savings** can be obtained compared with a linear page table
 - Unallocated pages (e.g. between heap and stack) no longer take up space in the page table
- A TLB may still be used to speed up address translation
 - On a TLB miss the segment bits determine which Base and Bounds pair to use for the page table

- **Disadvantages**

- **External fragmentation** may still arise as the page tables are of unequal size
- Segmentation assumes a certain usage pattern of the address space